Brownout & Controlled-Degradation Testing

in Release Readiness — Explained in Simple Terms

Components · Workflow · Architecture · Metrics · Worked Example

The original project statement this document explains

"Prevented production outages by driving brownout and controlled-degradation testing (dependency latency injection, traffic throttling, partial-failure simulation) in release readiness, with dashboards and alerts on error-budget burn rate, p95/p99 latency, saturation, and recovery time."

Contents

1. The project in plain English

1.1 One sentence

Before any new version of the software was allowed to go live, I deliberately made the system a little bit sick under safe, controlled conditions, watched exactly how it coped, and blocked the release if it did not cope well enough.

1.2 What the word “brownout” means here

The word is borrowed from electricity. A blackout is when the power goes off completely. A brownout is when the voltage dips: the lights go dim, the fridge struggles, the fan slows down — but nothing is technically “off”.

Software behaves the same way. A total outage (everything down) is actually rare. What is far more common is a brownout: the website still loads, but pages take four seconds instead of half a second, one button in ten fails, and the checkout page spins forever. Customers feel it immediately, but no monitor says “DOWN”, so nobody reacts quickly.

Almost every large outage starts life as a brownout. Something gets slow, the slowness backs up into the next component, that component runs out of room, and twenty minutes later the whole system has fallen over. The brownout is the warning shot.

1.3 What “controlled degradation testing” means

Instead of waiting for a real brownout to happen at 2 a.m. on a Saturday, I caused one on purpose — at a time I chose, in an environment real customers do not touch, at a dose I controlled, with a switch to stop it instantly.

The question being answered is never “does it work?” Ordinary testing already answers that. The question is: “when conditions get worse, does this version bend gracefully, or does it snap?”

> **The everyday analogy**
> A cardiologist does not decide you are healthy by watching you sit in a chair. They put you on a treadmill and gradually increase the speed while measuring your heart, your breathing and your blood pressure — and they stop the moment a reading crosses a safe limit.
> That is exactly this project. The treadmill is the injected fault. The heart monitor is the dashboards. The safe limit is the abort threshold. The verdict at the end decides whether the new release is fit to go live.

1.4 Why this belongs in “release readiness”

Release readiness is the checklist a new version has to pass before it is allowed into production. Most checklists ask: do the features work, do the tests pass, is the documentation ready. Those questions all assume the world around the software is behaving perfectly.

The world is not perfect. Databases get slow. Partner APIs time out. One data centre loses capacity. A new version can pass every functional test and still be the version that takes the site down, simply because someone set a timeout too high or removed a fallback. This project added the missing question to the checklist: how does this specific build behave when its surroundings misbehave?

1.5 What was actually built

- **A test harness —** a repeatable way to inject three kinds of stress (slowness, reduced capacity, and partial errors) into a copy of production;
- **A measurement layer —** four dashboards that show, live, whether the system is coping: how fast the reliability allowance is being spent, how slow the slowest requests are, how full the resources are, and how quickly things return to normal;
- **A safety layer —** automatic alerts and an automatic stop, so an experiment can never do real damage;
- **A decision layer —** a scorecard wired into the release pipeline that turns the results into a plain Go or No-Go answer.

The outcome in the original statement — “prevented production outages” — comes from finding the weak points in a test environment, on a Tuesday afternoon, with nobody watching, instead of finding them in production, on a Saturday night, with customers watching.

2. Plain-English glossary of every term used

Every term in the original statement, plus the ones needed to understand it. Nothing here assumes prior knowledge.

Term	What it means, in simple words
Outage	The system is unusable. The hard, obvious failure everyone notices.
Brownout	The system is still up but noticeably unwell: slow, or failing some of the time. Harder to spot, and usually the first stage of an outage.
Degradation	Any drop in quality of service — slower responses, some errors, fewer features working.
Graceful degradation	The good outcome. Under pressure the system gives up something small on purpose (skips a recommendation panel, serves slightly stale data) so the important part (checkout, login) keeps working.
Controlled degradation testing	Causing degradation on purpose, in a safe place, at a dose you choose, to see whether degradation is graceful or catastrophic.
Fault injection	The act of deliberately introducing a problem — a delay, an error, a capacity limit — into a running system.
Dependency	Anything the service needs in order to do its job: a database, a cache, a queue, another internal service, a payment provider.
Latency	How long a request takes. Measured in milliseconds (ms). 1,000 ms = 1 second.
Dependency latency injection	Deliberately adding a delay to the calls the service makes to one of its dependencies, to imitate that dependency being slow.
Traffic throttling	Deliberately limiting how much traffic is allowed through, or how much capacity the service has, to imitate a capacity shortage.
Partial-failure simulation	Making a portion of calls fail (say 20 %) while the rest succeed — imitating the messy half-broken reality, rather than a clean total failure.
Percentile (p50, p95, p99)	Sort every request from fastest to slowest. p50 is the middle one, p95 is the one 95 % are faster than, p99 is the one 99 % are faster than. p95 and p99 describe the unlucky users.
SLI	Service Level Indicator — the number you actually measure, e.g. “percentage of requests that succeeded”.
SLO	Service Level Objective — the target for that number, e.g. “99.9 % of requests must succeed”.
Error budget	The failure the SLO allows. If the target is 99.9 %, then 0.1 % of requests are allowed to fail. That 0.1 % is the budget you may spend.
Burn rate	How fast the error budget is being spent compared with plan. 1× is exactly on plan. 10× means a month’s allowance is being consumed in three days.
Saturation	How full a limited resource is: CPU, memory, thread pool, database connections, queue depth. Usually expressed as a percentage of capacity.

Term	What it means, in simple words
Recovery time	How long the system takes to return to normal after the problem is removed. Also called time-to-heal.
Steady state	The normal, healthy behaviour you measure first, so you have something to compare the experiment against.
Blast radius	How much can possibly be affected by an experiment — which service, which share of traffic, for how long. Kept deliberately small.
Kill switch	A single control that removes all injected faults instantly.
Canary	A small, isolated copy of the release used for testing before the full rollout.
Timeout	The maximum time a caller will wait for an answer before giving up.
Retry	Automatically trying a failed call again.
Retry storm	The trap where a slow dependency causes retries, the retries multiply the load, and the extra load makes it slower still.
Circuit breaker	A safety device in code: after too many failures it stops calling a sick dependency for a while, so the caller stops wasting resources waiting.
Bulkhead	Reserving separate pools of resources per dependency, so one sick dependency cannot consume every thread the service has.
Load shedding	Deliberately rejecting some requests quickly when overloaded, so the rest can be served properly.
Release readiness	The set of checks a new version must pass before it is allowed into production.

3. The problem this project solves

3.1 Ordinary testing checks the wrong thing

A normal test suite asks whether the code produces the right answer when everything around it is healthy. A normal load test asks whether the system copes with more traffic when everything around it is healthy. Both assume a perfect environment.

Real incidents almost never begin with a bug in the new feature or with too much traffic. They begin with a dependency behaving slightly worse than usual.

3.2 Slow is more dangerous than dead

This is the single most important idea in the whole project. If a database instantly refuses connections, the service finds out in a millisecond, gives up, and returns an error or a fallback. Unpleasant, but contained.

If the same database instead answers in three seconds rather than twenty milliseconds, something far worse happens. Every request that needs the database now occupies a thread for three seconds instead of twenty milliseconds — roughly one hundred and fifty times longer. The pool of available threads fills up. New requests queue. Queued requests time out at the front door. The client retries. Retries double the load on a dependency that was already struggling. Within minutes a service that was merely slow is completely unavailable, and so is every other service that called it.

> **Why this matters for a release**
>
> Two builds can be functionally identical and behave completely differently in this scenario. The difference is usually invisible in code review: a timeout raised from 1 s to 10 s, a retry loop added without backoff, a fallback quietly removed, a connection pool left at its default size.
>
> The only reliable way to find those differences before customers do is to make the dependency slow on purpose and watch.

3.3 Failure is a cliff, not a slope

Systems do not degrade smoothly. A service can be perfectly fine with an extra 100 ms of database latency, still fine at 200 ms, and completely collapsed at 260 ms — because that is the point at which the thread pool finally runs out. There is a cliff edge, and its exact position is unknowable from reading the code.

The purpose of ramping the fault gradually is to find where that cliff is, measure how far away it is from normal operating conditions, and make sure the gap is comfortable. That gap is the real headroom of the release.

3.4 What used to happen instead

- Weaknesses were discovered by customers, during real incidents, at the worst possible time.
- Post-incident reviews produced actions like “review our timeouts” that nobody could verify, because there was no way to reproduce the condition.
- Resilience features — circuit breakers, fallbacks, retry budgets — were written but never exercised, so nobody knew whether they actually worked. Several turned out to be misconfigured and had been doing nothing for months.
- “Is this release safe?” was answered by opinion and confidence, not by evidence.

4. Every component explained, end to end

The system is built in six layers. Each layer is described below with: what it is, what it does, why it has to exist, and what goes wrong without it. The same six layers appear as the six numbered bands of the architecture diagram in Section 6.

4.1 Layer 1 — The release pipeline (CI / CD)

This is the existing automated conveyor belt that takes code from a developer's commit to production. The testing described here is not a separate activity performed by a special team; it is a stage bolted onto this conveyor belt so that it happens on every release without anyone having to remember.

4.1.1 Build the release candidate

The exact version being considered for production is compiled and packaged, and given a version stamp. The critical rule is that the artefact tested must be byte-for-byte the artefact shipped. If you test one build and ship another, the results mean nothing.

4.1.2 Deploy to a pre-production or canary environment

The candidate is deployed to an environment that resembles production as closely as possible — same configuration, same dependency versions, comparable resource limits — but which no real customer is using. This is where the deliberate damage will be done.

The closer this environment is to production, the more trustworthy the results. If the test environment has one instance where production has forty, or a tiny database where production has a large one, the numbers will be misleading. Where a fully comparable environment was not possible, the results were treated as directional rather than absolute, and the same experiment was repeated later on a small production canary with a very small blast radius.

4.1.3 The release-readiness gate

The gate is an automated checkpoint near the end of the pipeline. It reads the scorecard produced by the experiments and returns one of three answers: Go, Go-with-conditions (for instance, ship but keep the new feature behind a flag), or No-Go. Because it is automated, it is consistent — it cannot be talked out of a No-Go because the release is urgent.

Without a gate, all of this is just information. With a gate, it is a control.

4.2 Layer 2 — The experiment control plane

This layer decides what to strain, how hard, and for how long. It is the brain of the exercise, and it is kept completely separate from the thing being tested so that a struggling service cannot interfere with the experiment controlling it.

4.2.1 The experiment catalogue

Every scenario is written down as a small configuration file and stored in version control alongside the code. A scenario file states the target dependency, the type of fault, the starting dose, the ramp steps, how long each step is held, the share of traffic affected, and the thresholds that will abort the run.

Storing scenarios as files rather than as manual steps gives four things: the same test runs identically every time; changes to a test are reviewed like code; a failed release can be re-tested against exactly the same conditions after a fix; and the catalogue becomes a written record of every failure mode the team has thought about.

- **Typical catalogue entries:** Payments API responds 300 ms slower than usual.

- Primary database read latency doubles for ten minutes.
- Cache is unavailable for 10 % of lookups.
- Service capacity is cut in half (imitating the loss of one availability zone).
- The recommendation service returns errors for one call in five.
- A downstream queue consumer stalls, so the queue grows.

4.2.2 The orchestrator

The orchestrator is the component that actually runs a scenario. It starts the experiment, applies each step of the ramp on schedule, records the exact start and stop time of every step, watches the abort conditions, stops everything when a limit is crossed or the time is up, and writes the results file.

The precise timestamps matter more than they might appear to. Every conclusion in the report is of the form “when we did X at time T, the system did Y”. Without accurate timing you cannot line the fault up against the metrics, and you are left guessing about cause and effect.

4.2.3 The load generator

A fault injected into an idle system proves nothing, because an idle system has infinite spare capacity. The load generator therefore produces realistic traffic throughout the experiment: a realistic mixture of endpoints, a realistic ratio of reads to writes, realistic payload sizes, and a realistic rate — typically normal peak load, and sometimes a busy-day multiple of it.

The traffic must be shaped like real traffic, not just voluminous. A flood of identical cheap requests will all be served from cache and will exercise nothing.

4.2.4 Safety guardrails

Guardrails are the rules that make deliberate damage safe. They are defined before the run starts, and enforced by machine rather than by human attention.

- **Blast radius —** one service, one dependency, one environment, a bounded share of traffic.
- **Time limit —** every experiment expires automatically, so a forgotten fault cannot linger.
- **Abort thresholds —** the numeric limits (for example, burn rate above 6×, or p99 above 1 s) that end the run immediately.
- **Kill switch —** one command, always available, that removes every injected fault at once and restores normal behaviour.
- **Scope tagging —** faults are only ever applied to traffic that is explicitly tagged as test traffic, never to a real customer’s request.
- **Announced window —** run in working hours, with the owning team notified, so a human is always available.

4.3 Layer 3 — The fault-injection layer

This is the only part of the whole system that changes behaviour. It is a small proxy that sits in the network path of every call the service makes to its dependencies — usually a sidecar container next to the service, or a feature of the service mesh. The application code is not modified in any way, which is essential: the build tested must be the build shipped, and adding test-only code would invalidate the entire exercise.

The proxy normally passes calls straight through and does nothing. On instruction from the orchestrator it starts interfering, in one of three ways.

4.3.1 Dependency latency injection — "make it slow"

The proxy accepts the service's call, holds the reply for an extra number of milliseconds, and then releases it. To the service, the dependency simply appears to have become slower. Nothing is broken; everything is just late.

Controls: which dependency, how much delay, how much random variation (jitter) around that delay, what share of calls are affected, and whether the delay applies to reads, writes, or both.

What this reveals:

- Whether timeouts are set correctly. A caller's timeout must be shorter than the timeout of whoever called it, or requests are abandoned upstream while resources are still held downstream.
- Whether the thread or connection pool is large enough to absorb the extra waiting, and how close to exhaustion it comes.
- Whether the circuit breaker actually opens — and at what point. A great many circuit breakers are configured to trip on errors only, and slowness produces no errors, so they never fire when they are needed most.
- Whether slowness in something unimportant (a recommendation panel, an analytics call) is allowed to slow down something critical (checkout). It very often is, and that is a serious defect.
- Whether retries make things better or worse.

4.3.2 Traffic throttling — "make it smaller"

Here the proxy limits how much can get through: a cap on requests per second, a cap on how many calls may be in flight at once, a smaller connection pool, or fewer running instances. The service is not slowed down and nothing errors — it simply has less room than it is used to.

This imitates the loss of an availability zone, a rate limit imposed by a partner, a noisy neighbour consuming shared capacity, or an autoscaler that cannot add instances fast enough.

What this reveals:

- Whether the service sheds load intelligently — rejecting some requests quickly and clearly so the rest are served properly — or whether it accepts everything and serves all of it badly.
- Whether queues stay bounded. An unbounded queue under capacity pressure grows until it consumes all memory, and every item in it is stale by the time it is processed.
- How fast autoscaling reacts, and whether it reacts before users notice.
- Whether the most valuable traffic is protected when capacity is short, or whether a health-check endpoint gets the same priority as a paying customer's checkout.

4.3.3 Partial-failure simulation — "make some of it fail"

The proxy makes a defined share of calls fail — returning an error code, closing the connection, timing out, or occasionally returning a malformed response — while the remainder succeed normally. This is the most realistic of the three, because real dependencies almost never fail cleanly and completely; they fail intermittently.

Controls: the share of calls affected, the type of failure, and whether failures come in bursts or are evenly scattered.

What this reveals:

- Whether the retry policy is safe. Retrying immediately, without exponential backoff and random jitter, turns a small problem into a self-inflicted flood.
- Whether fallback paths exist and actually work — serving slightly stale cached data, or hiding an optional panel, rather than failing the whole page.

- Whether retried write operations are idempotent. If not, a retry can double-charge a customer or create a duplicate order, which is far worse than an error message.
- Whether partial failures are visible. A common and dangerous finding is that a failure is silently swallowed by an empty catch block, so the user sees an empty screen and the dashboards see nothing wrong.
- Whether the system stays consistent when one step of a multi-step operation fails.

4.4 Layer 4 — The system under test

This is the release candidate itself plus everything it depends on. It is not modified for the test; the whole point is to observe the real build with its real configuration.

- **API gateway / edge —** the front door where requests arrive, where throttling and load shedding are applied.
- **Service under test —** the new build, running with production-like configuration: the same timeouts, pool sizes, retry policy and feature flags it will have in production.
- **Dependencies —** databases, caches, queues and their workers, internal services, and external partner APIs. Each is a candidate target for injection, and each has a different failure signature.

A point worth emphasising, and one shown explicitly in the diagram: the injection proxy sits between the service and its dependencies. The fault is applied to the conversation between them, not inside either one. That is what makes the test both realistic and instantly reversible.

4.5 Layer 5 — The observability pipeline

Injecting a fault is easy. The value comes entirely from measuring the response precisely. This layer turns raw behaviour into numbers that can be compared, charted and judged.

4.5.1 Metrics

Counters and timings emitted continuously by every component: requests received, responses by status code, duration histograms, pool usage, queue depth, CPU and memory. Metrics are cheap and complete, which makes them the backbone of the four dashboards. They are collected at a fine interval — ten to fifteen seconds — because averaging over a minute would smooth away exactly the spikes being hunted.

Latency is stored as a histogram rather than an average, because percentiles cannot be recovered from an average afterwards. This is a detail that is easy to get wrong and impossible to fix retrospectively.

4.5.2 Distributed tracing

A trace follows a single request across every service and dependency it touches, recording how long each hop took. Metrics tell you that requests got slower; traces tell you exactly which hop absorbed the time, and whether the delay was in the call itself or in waiting for a free connection to make the call. Traces are what turn “something is slow” into a specific line of code.

4.5.3 Logs

Structured logs supply the detail metrics cannot: which exception was thrown, whether a circuit breaker opened, whether a fallback path was taken, whether a retry was attempted. Logs from the experiment window are tagged with the experiment identifier so the relevant entries can be isolated from ordinary noise.

4.5.4 The time-series store

The database that keeps all these numbers over time and answers questions about them. It powers the dashboards, evaluates the alert rules, and holds the history that lets one release be compared with the previous one. Baselines

from previous runs are retained so that a slow drift in resilience — each release a little worse than the last — becomes visible.

4.6 Layer 6 — Dashboards, alerts and the decision

The four measurements are covered individually in Section 5. This subsection covers what sits around them.

4.6.1 Dashboards

Four purpose-built views, arranged so they can be read at a glance while a run is in progress. Each one shows the current run overlaid on the baseline, with the objective drawn as a horizontal line and the injection windows shaded, so “what changed and when” is obvious without needing to interpret anything.

4.6.2 Alert rules

Rules that watch the metrics and fire when something crosses a limit. During an experiment they serve a second purpose beyond notification: they are themselves under test. If a deliberate, obvious brownout does not trigger an alert, then the alerting is broken, and that is a finding in its own right — frequently one of the most valuable findings, because it means real brownouts have been going unnoticed.

Alerts are configured on multiple time windows: a fast window to catch a sudden severe problem, and a slower window to catch a mild problem that persists. This combination avoids both false alarms from brief spikes and blindness to slow bleeds.

4.6.3 Auto-abort

The same rules, wired to an action instead of a notification. When a guardrail is crossed, the orchestrator removes the fault immediately, without waiting for a human. Auto-abort is what makes it acceptable to push a system to its breaking point on purpose, and the fact that it is automatic is what makes the answer to “where is the cliff edge?” safe to obtain.

4.6.4 The readiness scorecard

A short generated report, attached to the release, recording for every scenario: the dose applied, the peak value of each of the four measurements, whether the guardrails held, the recovery time, and a pass or fail against the agreed criteria. It is the evidence the gate reads, and it is the artefact that turns a subjective judgement into a documented, reviewable decision.

5. The four measurements in detail

These four were chosen because together they answer four different questions: how much harm is being done, how bad it feels to a user, how close to the limit the system is, and whether it can heal itself.

Measurement	The question it answers	A bad result means
Error-budget burn rate	How much harm is this doing, in terms customers care about?	The release would consume the reliability allowance far faster than the business has agreed to.
p95 / p99 latency	How bad does this feel for the unluckiest users?	A significant number of real people would experience the product as broken, even though nothing is technically down.
Saturation	How close is the system to running out of room?	There is little headroom; a small extra push would tip it over.
Recovery time	Once the problem stops, can it heal by itself, and how fast?	An incident would outlive its cause and require manual intervention to end.

5.1 Error-budget burn rate

5.1.1 The idea, from the beginning

Perfect reliability is impossible and, in any case, absurdly expensive. So a target is agreed instead: for example, 99.9 % of requests must succeed. The remaining 0.1 % is not a failure of the team — it is an allowance, deliberately granted, and it is called the error budget.

Over thirty days, 0.1 % of the time is about forty-three minutes. That is the whole month's allowance for being unwell.

5.1.2 What burn rate adds

Burn rate converts "we are having some errors" into "how urgent is this". It is the observed failure rate divided by the allowed failure rate. If 0.1 % is allowed and 0.6 % is happening, the burn rate is 6×: the budget is being spent six times faster than planned.

Burn rate	Failure rate observed (against a 99.9 % target)	The month's budget would be gone in
1×	0.1 % — exactly on plan	30 days (as intended)
2×	0.2 %	15 days
6×	0.6 %	5 days
14.4×	1.44 %	About 2 days
100×	10 %	About 7 hours

This is why burn rate is the headline number. A raw error rate of "0.6 %" sounds tolerable to most people. "This release would exhaust a month of allowed failure in five days" does not, and it is the same fact.

5.1.3 How it is used in the experiment

The burn rate is watched continuously through the run. The dose at which it crosses the abort threshold is recorded as the release's breaking point. Two releases can then be compared directly: if the previous version tolerated 300 ms of injected delay before burning at 6× and the new one tolerates only 120 ms, the new version has lost resilience, and that regression is visible even though every functional test passed.

5.2 p95 and p99 latency

5.2.1 Why averages are actively misleading

Suppose one hundred requests are served: ninety-nine take 100 ms and one takes 10 seconds. The average is 199 ms, which looks excellent. But one customer in a hundred waited ten seconds, gave up, and possibly tried again — adding load. The average concealed the only thing that mattered.

Percentiles do not conceal it. Sort the requests from fastest to slowest: p95 is the request 95 % were faster than, and p99 is the request 99 % were faster than. They describe the experience of the unlucky, which is where dissatisfaction and retries come from.

5.2.2 Why one percent is a large number

One percent sounds negligible until it is multiplied out. At one million requests a day, p99 describes ten thousand requests every day. If p99 is three seconds, ten thousand people a day experience a three-second wait. At the scale of a busy service, the tail is not an edge case; it is a crowd.

5.2.3 Why both are tracked, not just one

They tell different stories, and the gap between them is itself informative.

- **p95 and p99 rise together:** The problem is widespread — typically genuine capacity exhaustion affecting everyone.
- **p99 rises sharply while p95 barely moves:** A minority of requests are hitting a specific trap: an exhausted connection pool, one slow shard, a cache miss path, a retry that fires only on certain calls. This is the classic early signature of a brownout, and it is invisible in an average.
- **p50 starts to rise:** The system is already saturated across the board and is close to failing entirely.

5.2.4 How it is used in the experiment

p95 and p99 are captured at baseline, at each step of the ramp, and throughout recovery. The key figures recorded are how much injected delay the service can absorb before p99 breaches its objective, and — importantly — whether p99 rises by more than the delay injected. If adding 100 ms of dependency delay raises p99 by 900 ms, the service is amplifying the problem, usually through queueing or retries. That amplification factor is one of the most diagnostic numbers the whole exercise produces.

5.3 Saturation

5.3.1 What it is

Saturation is how full a limited resource is, as a share of its capacity. Every one of these is a resource that can run out, and each produces a different failure.

Resource	What running out of it looks like
CPU	Everything slows down at once; scheduling delays appear across all endpoints.

Resource	What running out of it looks like
Memory	Aggressive garbage collection, long pauses, then the process is killed and restarts.
Thread pool / worker pool	Requests wait for a worker before any work begins. The most common cause of brownouts triggered by a slow dependency.
Database connection pool	Requests queue for a connection. Latency rises steeply while the database itself looks healthy and idle.
Queue depth	Work is accepted faster than it is processed. Backlog grows, and everything in it becomes stale.
Disk / network I/O	Reads and writes stall; often the hidden cause when CPU and memory both look fine.
File descriptors / sockets	New connections are refused outright, usually with a confusing error message.

5.3.2 Why it is the early-warning signal

Saturation moves before latency does. A thread pool goes from 40 % to 95 % full while response times still look acceptable, because there is still just enough capacity. Then it reaches 100 %, and latency does not rise gently — it explodes, because requests are now queueing for a resource that has none left. Watching saturation gives warning during the flat part of the curve, before the cliff.

5.3.3 Why it identifies the true bottleneck

When a service is slow, saturation shows where the pressure has actually landed. If CPU is at 30 % and the database pool is at 100 %, adding more CPU or more instances will not help; the pool is the constraint. Without saturation data, teams routinely spend money scaling the wrong thing.

5.3.4 How it is used in the experiment

Saturation of every constrained resource is watched throughout. Two things are recorded: the peak reached at each dose, and how much headroom remained at the point where the run was aborted. A release that survives the test with 98 % pool utilisation has technically passed but has no margin at all, and is treated as a conditional pass at best.

5.4 Recovery time

5.4.1 What is measured

The clock starts the instant the injected fault is removed and stops when latency, error rate, saturation and queue depth are all back within their baseline range. It is measured from data, not estimated.

5.4.2 Why it deserves equal billing with the other three

Surviving pressure and recovering from it are two different abilities, and a system can have the first without the second. Common reasons a system fails to heal on its own once the cause is gone:

- A queue built up a large backlog during the incident, and clearing it takes far longer than the incident itself.
- Retries queued during the problem all fire at once the moment things improve, immediately overwhelming the recovering dependency — a second outage caused by the recovery from the first.
- A circuit breaker opened correctly but never closes again, because its half-open probe is misconfigured, so the dependency stays cut off long after it is healthy.

- Caches were emptied during the disruption, so every request now goes to the database, keeping the system slow for as long as it takes the cache to refill.
- A connection pool holds onto dead connections and never renews them.
- A process ran out of memory, restarted, and lost warm state — leaving it slow for several minutes after every restart.

5.4.3 Why it is the number that predicts outage length

Incident duration is not determined by how long the cause lasted. It is determined by how long the cause lasted plus how long the system takes to heal. A dependency that was slow for two minutes can produce a forty-minute incident if recovery is poor. Measuring recovery time on every release keeps that second term under control, and gives the on-call team a realistic expectation of how long things will take once a cause is fixed.

5.4.4 How it is used in the experiment

A target is agreed — for instance, full recovery within five minutes of the fault being removed, with no human intervention. The words “no human intervention” are the important part: if recovery requires someone to restart a service or clear a queue by hand, the release fails this criterion regardless of how quickly a human could theoretically do it, because at three in the morning nobody does anything quickly.

6. Architecture diagram and a full walk-through of it

The diagram below shows the whole system as six stacked layers, read from top to bottom, with two feedback loops running up the sides. Every box and every arrow is explained in the walk-through that follows it.

Figure 1 — Brownout & Controlled-Degradation Testing: End-to-End Architecture

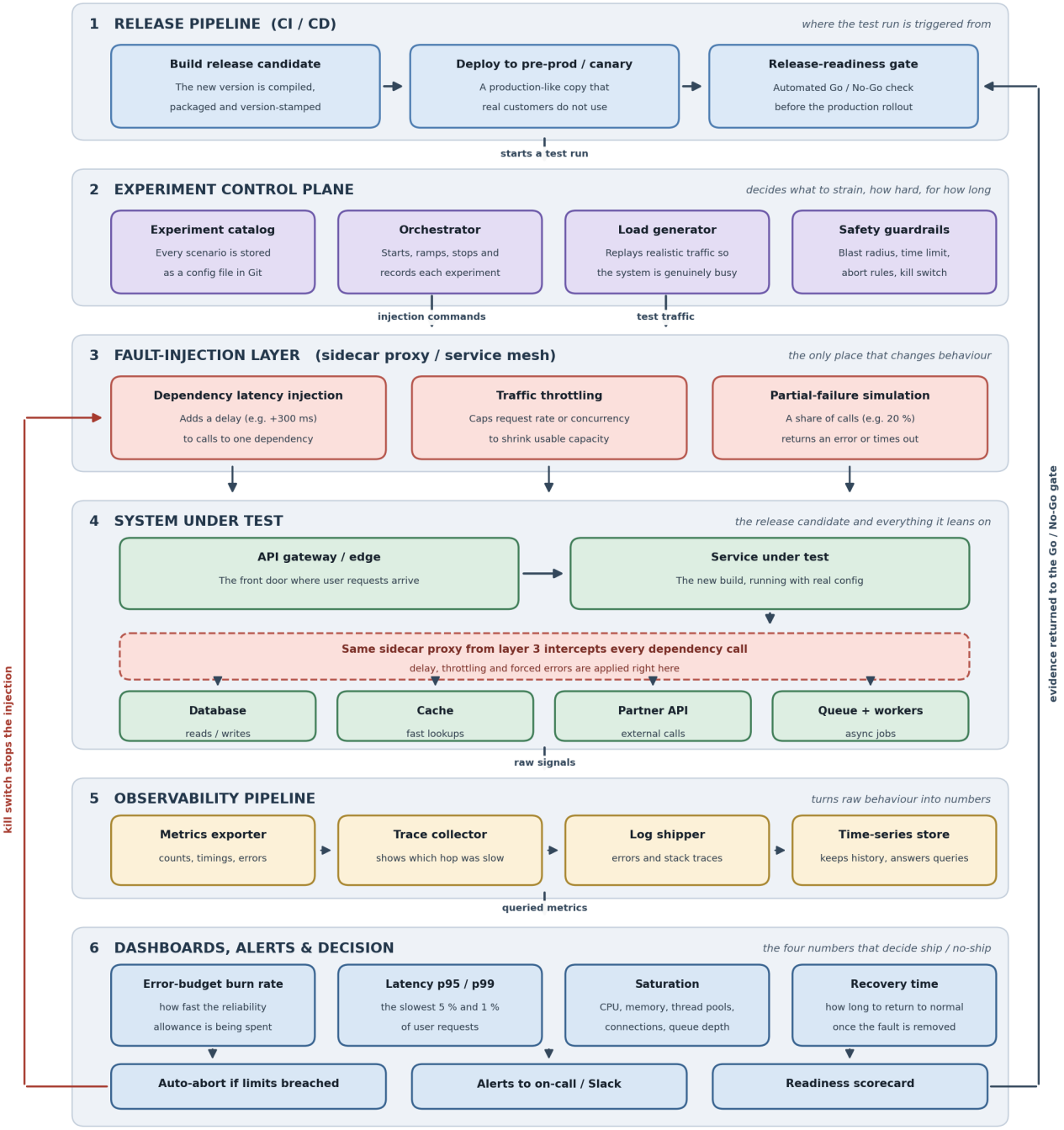

Figure 1 — The complete architecture. Solid arrows show the forward flow of a test run; the two coloured lines down the sides are the feedback loops.

6.1 How to read the diagram

- Each numbered grey band is one layer of the system, and they are stacked in the order things happen.
- Arrows pointing downwards are the forward flow of a test run — a release triggers a test, the test injects faults, the faults produce behaviour, the behaviour produces numbers, the numbers produce a decision.
- The dark line down the right-hand side is the decision feedback loop: results travel back up to the release gate in layer 1.
- The red line down the left-hand side is the safety feedback loop: the kill switch reaches straight back into layer 3 and removes the fault.
- The dashed red strip inside layer 4 is not a separate component. It is the same proxy from layer 3, drawn again in the position where it physically sits — between the service and its dependencies.

6.2 Band 1 — Release pipeline

Three boxes, left to right, along the top. Build release candidate produces the exact artefact under consideration. Deploy to pre-prod / canary places it in a production-like environment no customer is using. Release-readiness gate is the checkpoint the whole exercise exists to inform.

The arrow leaving the middle box downwards, labelled “starts a test run”, is the trigger: as soon as a candidate is deployed, the experiments begin automatically. Nobody has to remember to run them, which is the difference between a practice that survives and one that quietly lapses after two months.

6.3 Band 2 — Experiment control plane

Four boxes. The Experiment catalogue holds the scenario definitions as version-controlled files. The Orchestrator executes a chosen scenario, ramps it, times it and records it. The Load generator keeps realistic traffic flowing so the system is genuinely busy. Safety guardrails hold the blast radius, the time limit, the abort thresholds and the kill switch.

Two arrows leave this band. “Injection commands” goes from the orchestrator down to layer 3, telling it what fault to apply. “Test traffic” goes from the load generator down through the layers to the system itself. They are drawn separately on purpose, to make clear that the traffic path and the control path are different things: the control plane never touches customer requests.

6.4 Band 3 — Fault-injection layer

Three boxes, one for each technique named in the project statement, and all three live in the same proxy.

- **Dependency latency injection —** holds replies for an extra number of milliseconds, making a dependency appear slow.
- **Traffic throttling —** caps request rate or concurrency, shrinking the room the service has to work in.
- **Partial-failure simulation —** makes a defined share of calls return errors or time out, leaving the rest working.

The band caption reads “the only place that changes behaviour”, and that is a deliberate design constraint. Because nothing else is modified, removing the fault restores the system exactly, instantly, with no deployment and no restart.

6.5 Band 4 — System under test

The top row shows the request path: the API gateway is the front door, and the arrow to its right leads into the service under test — the new build, running with real configuration.

Below the service is the dashed red strip: the injection proxy, intercepting every dependency call. The four boxes underneath it are the dependencies — database, cache, partner API, and queue with its workers — each reached through the proxy, which is why each has an arrow arriving from the strip rather than from the service directly.

This is the most important geometric fact in the diagram. The fault is applied to the conversation between the service and a dependency. That is precisely where real-world degradation appears, and it is why the technique is realistic without requiring the dependency itself to be damaged.

6.6 Band 5 — Observability pipeline

The arrow labelled “raw signals” carries the system’s behaviour into this band. The Metrics exporter contributes counts and timings; the Trace collector shows which hop absorbed the time; the Log shipper supplies exceptions and evidence of fallbacks and breakers. All three feed the Time-series store, which keeps the history and answers the queries.

The three small arrows between these boxes indicate that everything converges on the same store, so that a single query can combine what happened, where it happened and why.

6.7 Band 6 — Dashboards, alerts and decision

The arrow labelled “queried metrics” feeds the four dashboards on the upper row: error-budget burn rate, p95 / p99 latency, saturation, and recovery time. These four are the entire basis of the verdict.

Beneath them sit the three things the dashboards drive. Auto-abort ends the experiment when a guardrail is crossed. Alerts notify the on-call engineer and the team channel — and are themselves being tested. The Readiness scorecard records the outcome.

6.8 The two feedback loops

The dark line on the right carries the scorecard from band 6 back up to the release-readiness gate in band 1. Without it the exercise would only produce interesting charts; with it, the results have authority over whether the release proceeds.

The red line on the left runs from auto-abort back into the fault-injection layer. It is drawn as a direct path, bypassing every intermediate layer, because that is exactly how it works: the stop signal does not queue behind anything. Nothing has to be redeployed, nothing has to be restarted, and no permission is needed. That single property is what makes it safe to deliberately push a system to its breaking point.

7. The workflow, step by step, end to end

Figure 2 summarises the ten steps of a single run, grouped into four phases. Each step is then explained in full underneath.

Figure 2 — The End-to-End Workflow of a Single Brownout Test Run

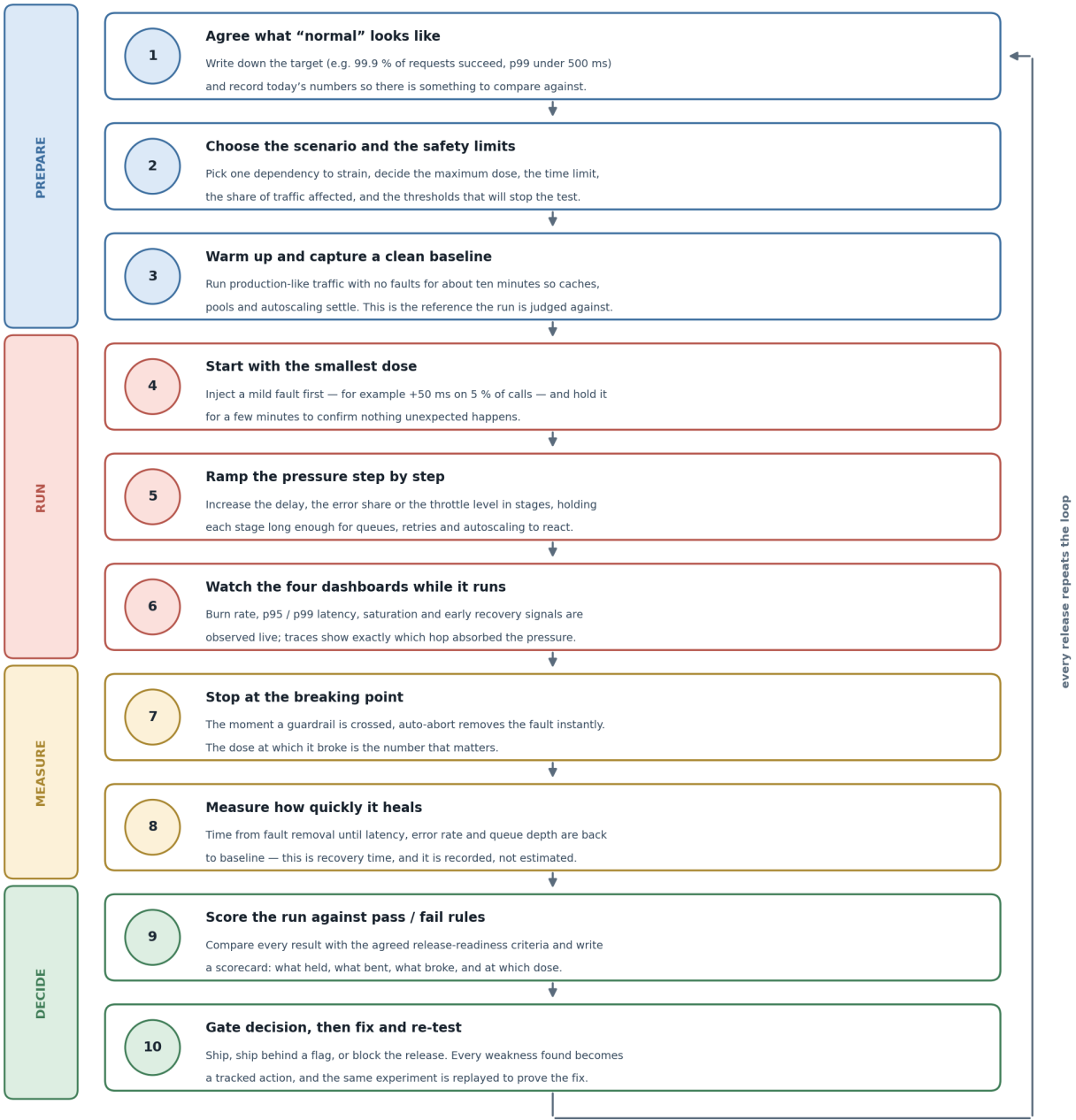

Figure 2 — The ten steps of one experiment, from defining normal to deciding whether to ship.

Step 1 — Agree what “normal” looks like

Nothing in this exercise means anything without a definition of normal, so this comes first and it is written down.

- Set the objective: for example, 99.9 % of requests succeed and p99 stays under 500 ms.
- Record current behaviour under ordinary load: p50, p95 and p99 latency, error rate, saturation of every constrained resource, queue depths.
- Agree the abort thresholds that will end a run, and the pass / fail criteria that will judge it. Agreeing these in advance is what stops results being rationalised after the fact.
- Identify which dependencies are critical (the request cannot succeed without them) and which are optional (the request can succeed in a reduced form). This distinction drives most of the later analysis.

Step 2 — Choose the scenario and the safety limits

One scenario is selected from the catalogue. Changing one thing at a time is essential; two simultaneous faults produce results nobody can attribute.

- Choose the target dependency and the fault type — slowness, reduced capacity, or partial errors.
- Set the ramp: the starting dose, the size of each increase, and how long each step is held. Steps are held for several minutes, because autoscaling, queues and retry backoffs all react on that timescale.
- Set the blast radius: which environment, which share of traffic, which service.
- Set the hard time limit, after which the experiment expires by itself.
- Confirm the kill switch works before starting. This is checked every single time, not assumed.

Step 3 — Warm up and capture a clean baseline

Production-like traffic is run for roughly ten minutes with no faults at all, so that caches fill, connection pools reach their steady size, just-in-time compilation settles and autoscaling stabilises.

- The numbers from this window become the reference for the whole run.
- A cold system looks unhealthy for reasons that have nothing to do with the experiment; skipping the warm-up is the single most common way to produce a meaningless result.
- If the baseline itself is already outside the objective, the run is stopped here — there is no point stressing something that is unwell to begin with, and that fact is itself a finding.

Step 4 — Start with the smallest dose

The first injection is deliberately mild, for instance +50 ms of delay on 5 % of calls to one dependency, held for a few minutes.

- A small dose confirms the mechanism is working and affecting the intended target, and that observability is capturing it.
- It also confirms the abort path is live, before there is anything at stake.
- At this dose a healthy system should show almost no user-visible change. If a mild dose already moves p99 noticeably, that is a significant finding on its own and the run may end here.

Step 5 — Ramp the pressure step by step

The dose is increased in stages — more delay, a larger share of calls, a tighter throttle — with each stage held long enough for the system to fully react.

- Ramping rather than jumping is what locates the cliff edge. Jumping straight to a severe fault tells you only that severe faults break things, which was never in doubt.
- Each step is timestamped so it can be lined up precisely against the metrics.
- The point of interest is not where things break but where behaviour stops being linear: the first dose at which the response is disproportionate to the input.

Step 6 — Watch the four dashboards while it runs

Live observation during the run catches things a post-run report cannot, because sequence and timing carry information that summary statistics discard.

- Burn rate: is real customer-visible harm being caused, and how fast?
- p95 / p99: is the tail rising faster than the injected delay — that is, is the system amplifying the problem?
- Saturation: which resource is filling up first? That resource is the true bottleneck.
- Traces: which hop absorbed the time, and did it wait on the call itself or on getting a connection to make the call?
- Logs: did the circuit breaker open, was the fallback taken, did retries fire, and were they backed off?
- Downstream traffic: this is where a retry storm becomes visible — the load leaving the service rises while the load arriving stays flat.

Step 7 — Stop at the breaking point

The moment a guardrail is crossed, auto-abort removes the fault instantly. The dose at which that happened is the headline result of the run.

- The fault is removed completely and at once, not tapered — both because that is safer and because a clean edge makes the recovery measurement exact.
- The abort is recorded with its trigger, its exact time and the dose in force.
- If the ramp reaches its planned maximum without any guardrail being crossed, that is the best possible outcome: the release has been proven to tolerate more than the worst case anyone could think of.

Step 8 — Measure how quickly it heals

The clock runs from the instant of fault removal until every measurement is back inside its baseline range, with no human intervention of any kind.

- Latency must return to baseline, not merely improve.
- The error rate must return to baseline — a persistent trickle of errors after the cause is gone usually means a circuit breaker that will not close.
- Queues must drain, and the drain rate is recorded, since a backlog that clears in ninety minutes is effectively a ninety-minute outage.

- Saturation must fall back, and connection pools must renew rather than hold dead connections.
- Any manual step required is recorded as a failure of this criterion, however easy that step might be.

Step 9 — Score the run against pass / fail rules

Results are compared with the criteria agreed in step 1, and a scorecard is generated automatically and attached to the release.

- Every measurement is recorded per dose, so the shape of the degradation curve is preserved, not just its endpoint.
- Results are compared with the previous release to catch a slow erosion of resilience over successive versions.
- Every weakness found becomes a tracked action with an owner, rather than a paragraph in a report nobody revisits.
- The scorecard states plainly what held, what bent, what broke, and at exactly which dose each happened.

Step 10 — Gate decision, then fix and re-test

The gate reads the scorecard and returns one of three answers.

- Go — the release absorbed everything thrown at it with margin to spare.
- Go with conditions — ship, but behind a feature flag, or with a lowered traffic cap, or with a specific alert added and a fix committed to the next cycle.
- No-Go — a critical criterion failed. The release is blocked, the weakness is fixed, and the identical experiment is replayed to prove the fix, which is straightforward precisely because the scenario is a file rather than a memory.
- Both the decision and its evidence are recorded, so the reasoning is available to whoever asks about it three months later.

8. A worked example of one complete run

The chart below shows a single sixty-minute experiment on a checkout service, with delay injected into its calls to the payments dependency. It is the clearest way to see all four measurements behaving at once.

Figure 3 — What One Brownout Run Looks Like on the Dashboards

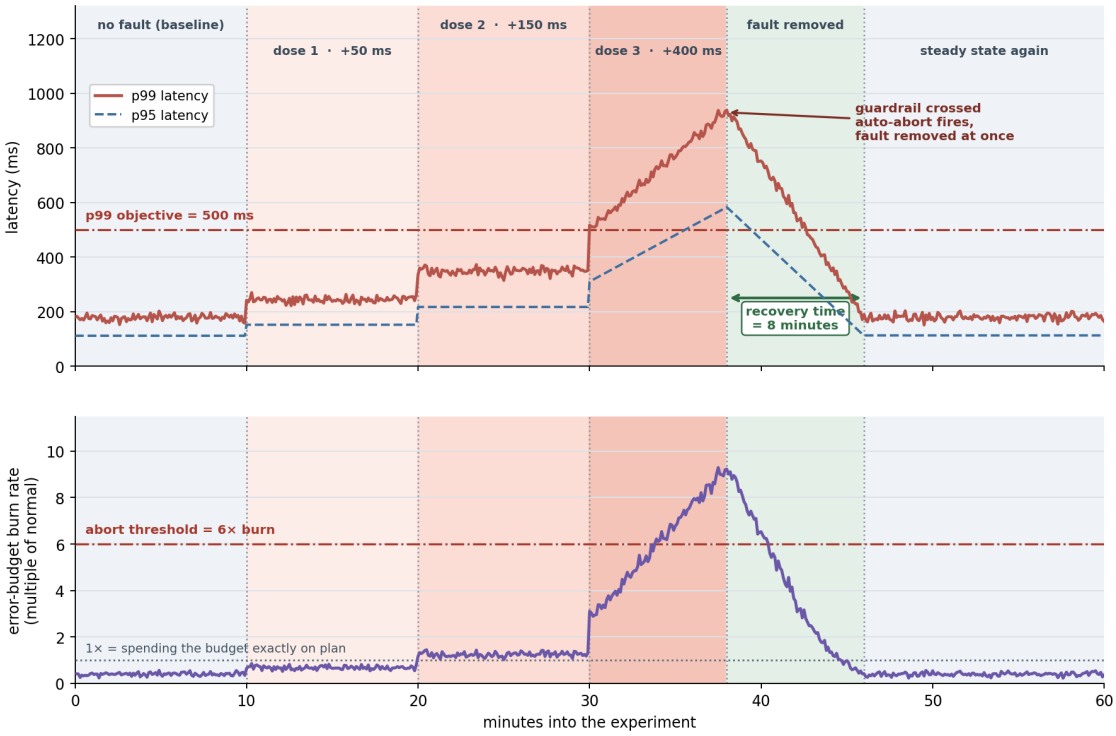

Figure 3 — One run: three increasing doses, an automatic abort, and the recovery that follows.

8.1 What happens, minute by minute

Minutes	What was done	What the system did
0 – 10	No fault. Realistic traffic only.	Baseline established: p99 about 180 ms, p95 about 110 ms, burn rate 0.4×. This is normal.
10 – 20	Dose 1: +50 ms delay on calls to payments.	p99 rises to roughly 245 ms — about the delay injected, and no more. The system is absorbing the fault cleanly. Burn rate barely moves.
20 – 30	Dose 2: +150 ms delay.	p99 reaches about 350 ms. Note that it rose by 105 ms for an extra 100 ms of delay — still close to linear, still healthy. Burn rate edges above 1×.
30 – 38	Dose 3: +400 ms delay.	Behaviour stops being linear. p99 climbs past the 500 ms objective and keeps climbing to about 930 ms — far more than the 250 ms of extra delay applied. The connection pool is saturating and requests are queueing for a connection. Burn rate accelerates past 6×.

Minutes	What was done	What the system did
38	Guardrail crossed. Auto-abort fires and the fault is removed instantly.	This is the breaking point: it lies between +150 ms and +400 ms of injected delay.
38 – 46	No fault. The system is left alone to recover.	Latency and burn rate fall steadily back to baseline over eight minutes, with no human intervention. Recovery time = 8 minutes.
46 – 60	Still no fault. Observation continues.	Everything holds at baseline, confirming the recovery is genuine and not a temporary dip.

8.2 What this run actually tells you

- The service tolerates 150 ms of extra payments latency comfortably, and breaks somewhere before 400 ms. Real payment-provider slowdowns of 200–300 ms are common, so the margin is thinner than anyone had assumed.
- The response is non-linear above 150 ms: 250 ms of extra delay produced almost 600 ms of extra p99. The service is amplifying the problem, not merely passing it on — the signature of queueing for an exhausted pool.
- The alerting worked. Burn rate crossed the threshold and the alert fired, which confirms that a real event of this shape would be noticed.
- Recovery took eight minutes with no intervention. That is honest, measured behaviour rather than an assumption — but it exceeds a five-minute target, so it is a finding.
- The likely fix is a larger connection pool, or a shorter timeout on payments combined with a circuit breaker that trips on slowness rather than only on errors. After the fix, this identical run is replayed, and the two curves are compared directly.

> **The sentence that makes this worth doing**
>
> “This release starts to degrade badly when payments get more than about 200 ms slower, and it takes eight minutes to recover.”
>
> That is a specific, measured, comparable statement about a specific build. Before this project existed, the equivalent statement was “it should be fine”.

9. Release-readiness pass / fail criteria

These are agreed in advance and applied mechanically. Fixing thresholds before the run is what prevents a marginal result from being talked into a pass because the release is urgent.

Criterion	Passes if	Fails if
Graceful behaviour	Under injected stress the service sheds optional work and protects the critical path.	Optional dependencies are allowed to slow or break the critical path.
Latency tail	p99 stays within its objective at the expected worst-case dose, and rises roughly in proportion to the injected delay.	p99 breaches its objective at a realistic dose, or rises far more than the delay applied.
Burn rate	Stays below the agreed multiple at the expected worst-case dose.	Crosses the abort threshold at a dose that could plausibly occur in production.
Saturation headroom	No constrained resource exceeds its agreed ceiling; there is visible margin left.	A resource reaches 100 %, or finishes the run with almost no headroom.
Recovery	Full return to baseline within the agreed time, with no human action.	Recovery takes longer than the target, or needs a restart, a manual queue purge, or any other intervention.
Retry safety	Retries are bounded, backed off, jittered, and write operations are idempotent.	Downstream load rises while inbound load is flat — the fingerprint of a retry storm.
Observability	The injected fault is clearly visible on the dashboards and triggers the expected alert.	A deliberate, obvious brownout produces no alert, or is invisible on the dashboards.
Comparison with previous release	Resilience is equal to or better than the previous version.	The new version breaks at a lower dose than its predecessor — a resilience regression.

10. Weaknesses this testing typically finds, and the fixes

These are the recurring findings. Every one of them is invisible to functional testing, and every one of them has caused real outages somewhere.

What is found	Why it is dangerous	The usual fix
Timeouts longer than the caller's timeout	The caller gives up while the service keeps holding a thread, so resources stay locked for work nobody is waiting for any more.	Set timeouts as a budget that shrinks at every hop down the chain.
Retries with no backoff or jitter	A brief wobble becomes a self-inflicted flood as every client retries in unison.	Exponential backoff with random jitter, a cap on attempts, and an overall retry budget.
Circuit breaker that trips only on errors	Slowness produces no errors, so the breaker never opens during the exact scenario it was written for.	Trip on latency and on concurrent-call count as well as on error rate.

What is found	Why it is dangerous	The usual fix
Undersized connection or thread pool	Requests queue for a connection; latency explodes while the dependency itself looks idle and healthy.	Resize the pool, and add bulkheads so one dependency cannot consume every worker.
Optional dependency on the critical path	A recommendation panel or an analytics call takes down checkout.	Make it genuinely optional: short timeout, fallback, and never block the response on it.
Unbounded queue	The backlog grows until memory runs out, and everything in it is stale by the time it is processed.	Bound the queue, shed load when it is full, and alert on depth and on age.
Non-idempotent retried writes	A retry double-charges a customer or creates a duplicate order.	Idempotency keys, and de-duplication at the receiving end.
Silently swallowed failures	The user sees an empty screen and the dashboards see nothing at all.	Log, count and surface every failure; alert on the fallback path being used.
Autoscaling too slow to matter	Capacity arrives several minutes after the users have already left.	Lower the trigger threshold, pre-scale ahead of known peaks, keep warm capacity.
Cache stampede after expiry	Everything expires at once and the whole load lands on the database in a single instant.	Staggered expiry, single-flight refresh, and serve stale while revalidating.
Recovery requires a manual restart	The incident lasts until someone wakes up, regardless of how quickly the cause was fixed.	Self-healing: health-check-driven restarts, automatic breaker reset, automatic backlog drain.
Missing alert for degraded states	Brownouts go unnoticed until they become outages, which is how most large incidents begin.	Multi-window burn-rate alerts plus alerts on p99 and on saturation, not only on hard failure.

11. Safety rules that keep the testing itself harmless

Deliberately degrading a system is only responsible if it cannot cause the harm it is meant to prevent. These rules are non-negotiable and are enforced by the tooling rather than by discipline.

- **Pre-production first —** Every scenario runs in pre-production first. Production is only ever considered afterwards, on a tiny canary, for scenarios already proven safe.
- **Smallest possible blast radius —** One service, one dependency, a bounded share of traffic, one environment. Never two faults at once.
- **Hard time limits —** Every experiment expires automatically. A forgotten fault cannot outlive the session that created it.
- **A tested kill switch —** One control removes everything instantly, and it is verified before each run rather than assumed to work.
- **Automatic abort —** Numeric thresholds stop the run by machine, without waiting for a person to notice or decide.
- **Announced windows —** Run during working hours, with the owning team notified in advance and a human available throughout.
- **Tagged traffic only —** Faults are applied only to traffic explicitly marked as test traffic. A real customer request is never a target.
- **No real data —** Synthetic data only. No real customer information is used in any scenario.
- **Full audit trail —** Every run is logged with its scenario, dose, timing, operator and outcome, so anything unusual can be traced back to it.

12. How this actually prevents production outages

The claim in the original statement is that outages were prevented. The mechanism is worth stating precisely, because “prevented an outage” can sound unfalsifiable.

Mechanism	What it changes
Weaknesses are found before release, not during an incident	Each misconfigured timeout, missing fallback or undersized pool found in a test environment is one that cannot cause an incident in production.
Resilience features are proven rather than assumed	Circuit breakers, fallbacks and retry budgets are exercised on every release. Several are always found to be misconfigured and doing nothing at all.
The cliff edge is a known number	The team knows how much dependency slowdown the service tolerates. Capacity planning, timeout settings and alert thresholds all become decisions based on evidence rather than instinct.
Alerting is validated on every release	A deliberate brownout that produces no alert reveals a monitoring gap. Real brownouts are then caught early, while they are still small and cheap to fix.
Recovery time is measured, not hoped for	Incident duration stops being open-ended. Where recovery is slow, it is fixed — which shortens every future incident of that shape, whatever caused it.

Mechanism	What it changes
Resilience regressions are caught between releases	Comparing each release with the last catches the gradual erosion where every version is slightly less robust than the one before, which is otherwise invisible until it is severe.
The go / no-go decision becomes evidence-based	"It should be fine" is replaced by a scorecard. Risky releases are held back or shipped behind a flag, deliberately rather than accidentally.

13. One-page recap

- **The idea —** A brownout is when a system is unwell but not down: slow, or failing part of the time. It is how most outages begin.
- **The method —** Cause a brownout deliberately, in a safe environment, at a dose you control, before a release goes live — and watch what happens.
- **The three levers —** Make a dependency slow (latency injection), give the service less room (traffic throttling), and make part of its calls fail (partial-failure simulation).
- **The four measurements —** How fast the reliability allowance is spent (burn rate), how bad it feels for the unluckiest users (p95 / p99), how full the resources are (saturation), and how quickly it heals afterwards (recovery time).
- **The workflow —** Define normal, choose a scenario, take a baseline, inject a small dose, ramp it, watch, abort at the limit, measure the recovery, score it, and let the gate decide.
- **The safety net —** Blast radius limits, automatic time limits, automatic abort, and a kill switch that is tested before every run.
- **The result —** Weak points are found on a quiet weekday afternoon in a test environment, instead of on a Saturday night in production with customers watching.

> **If you have to explain the whole thing in thirty seconds**
>
> "Most outages don't start as outages — they start as something getting a bit slow, and the system reacting badly to that. So before every release I deliberately made a dependency slow, or cut the service's capacity, or made some of its calls fail, in a safe environment. I measured four things while it happened: how fast we were burning the error budget, what the slowest one percent of users experienced, how full the resources got, and how long it took to recover once I stopped. If the release couldn't bend without breaking, it didn't ship until it could."